

Content Hub

Architecture & Design Document

Version 1.0 · June 2026 · System Design Reference

Contents

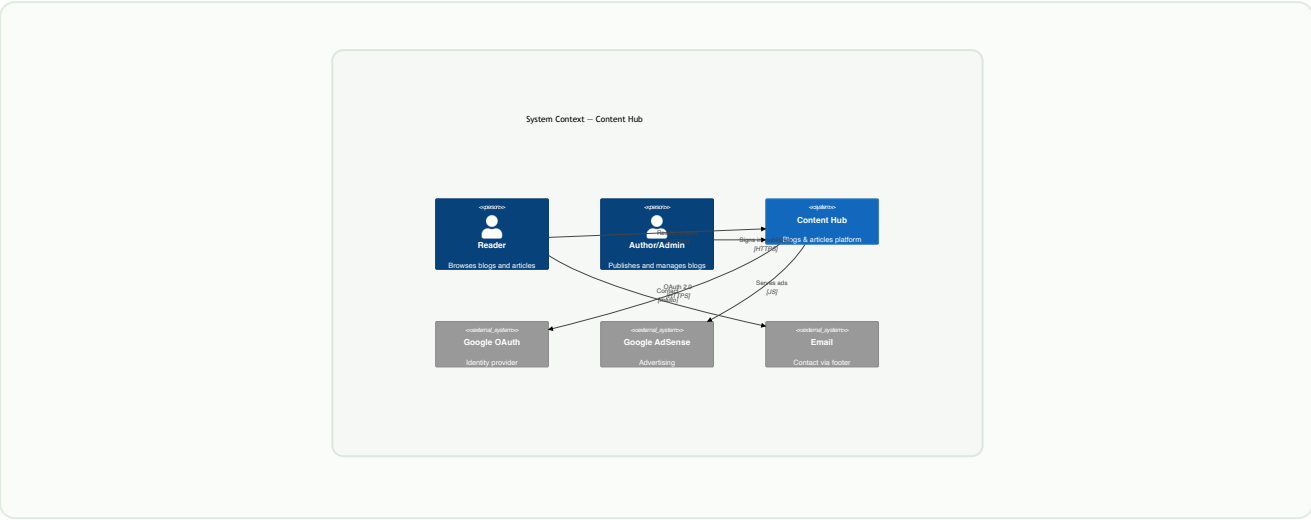
1. Executive Summary
2. System Context (C4 Level 1)
3. Container Architecture (C4 Level 2)
4. Deployment Architecture
5. Data Model
6. Security Architecture
7. Authentication Design
8. Content & Blog Design
9. API Design Principles
10. Non-Functional Requirements
11. Design Decisions

1. Executive Summary

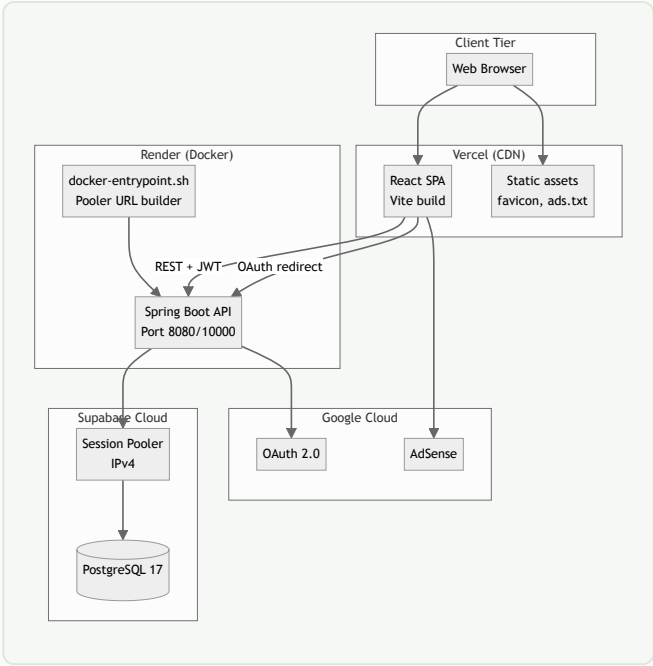
Content Hub is a three-tier web application delivering a public content portal (blogs and articles) with an authenticated admin publishing workflow. The architecture separates static UI delivery (Vercel CDN), stateful API logic (Render containers), and managed persistence (Supabase PostgreSQL).

Goal	Approach
Fast global UI	Static React SPA on Vercel edge CDN
JVM backend without PaaS Java support	Docker on Render
Managed PostgreSQL	Supabase with Flyway migrations
Flexible auth	JWT + optional Google OAuth2
Cost-effective MVP	Free tiers: Vercel, Render, Supabase

2. System Context (C4 Level 1)



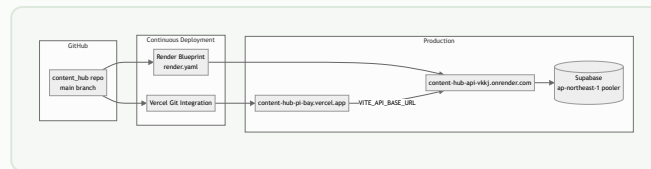
3. Container Architecture (C4 Level 2)



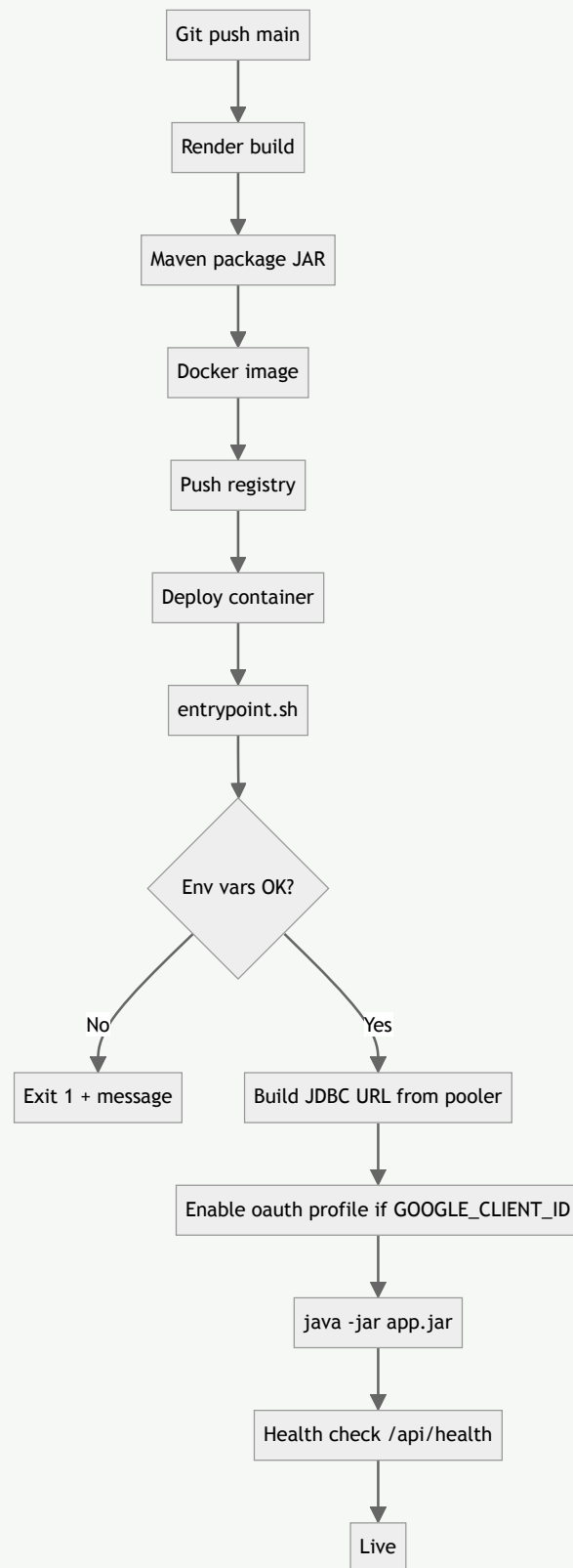
Component responsibilities

Container	Responsibility	Technology
React SPA	UI, routing, client state, API calls	React 19, Vite
Spring Boot API	Auth, validation, business rules, REST	Java 17, Spring 3.4
PostgreSQL	Users, articles, blogs persistence	Supabase
Docker image	Reproducible JVM runtime on Render	Eclipse Temurin 17

4. Deployment Architecture



Render Docker pipeline

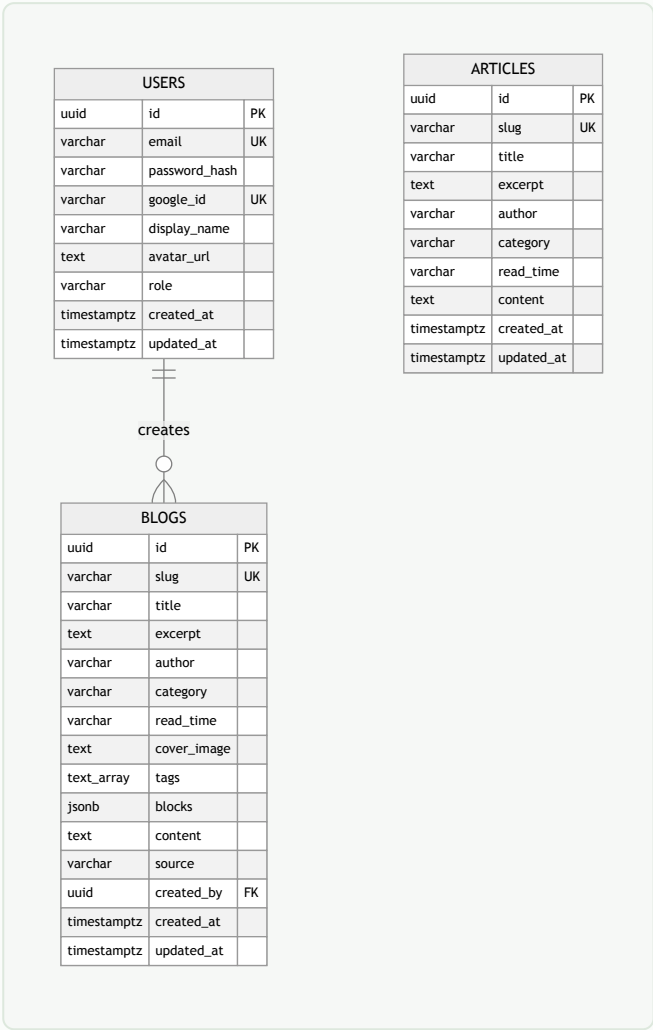


Network & connectivity

Path	Protocol	Notes
Browser → Vercel	HTTPS	Static SPA, CDN cached
Browser → Render API	HTTPS	CORS with credentials
Render → Supabase	PostgreSQL SSL	Session pooler (IPv4 required)
Render → Google	OAuth 2.0 HTTPS	Redirect URI on Render host

Render free tier is IPv4-only. Supabase direct `db.*.supabase.co` is IPv6 — use session pooler `aws-1-ap-northeast-1.pooler.supabase.com`.

5. Data Model



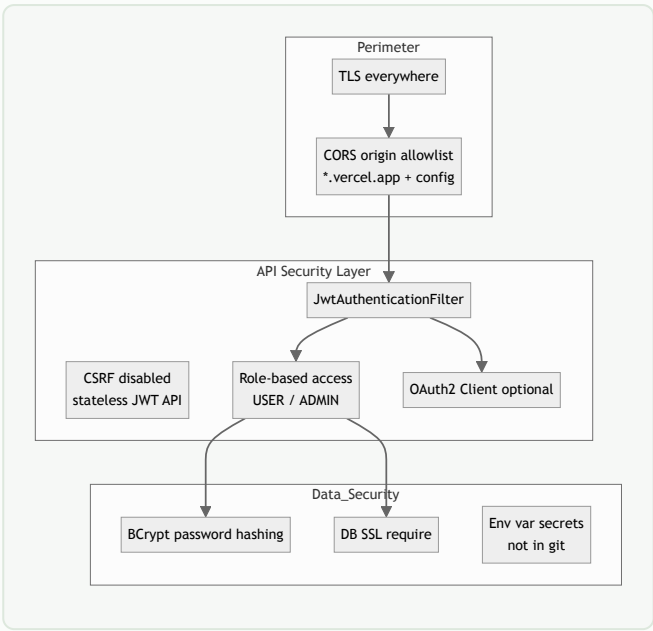
Blog source types

source	Origin	Editable in Admin?
json	seed/blogs.json via DataSeeder	No (read-only in UI)
admin	AdminPanel form submission	Yes (delete supported)
api	Default for API-created posts	Yes

Schema migration strategy

Flyway manages schema via `v1__init_schema.sql`. `baseline-on-migrate: true` allows coexistence with manually applied Supabase SQL. Hibernate `ddl-auto: validate` ensures entity/schema alignment at runtime.

6. Security Architecture



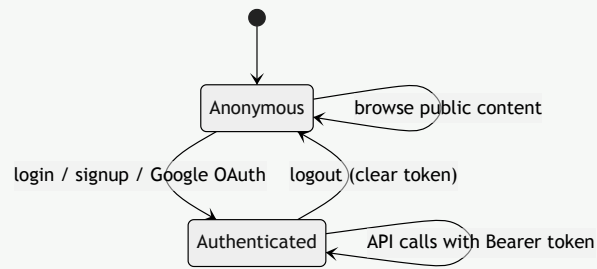
Endpoint access matrix

Resource	Anonymous	Authenticated USER	ADMIN
GET articles/blogs	✓	✓	✓
POST /api/blogs	✗	✓	✓
DELETE /api/blogs	✗	✓	✓
GET /api/auth/me	✗	✓	✓

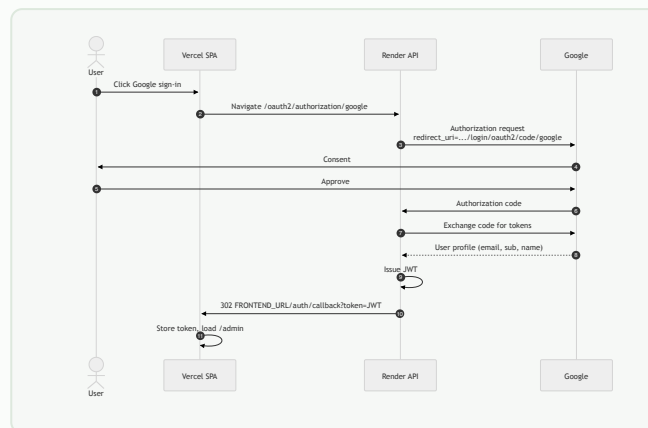
7. Authentication Design

JWT token structure

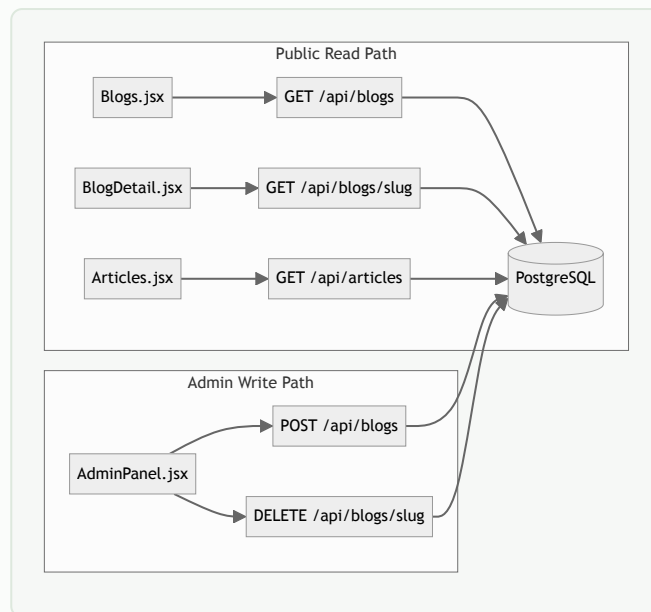
Claim	Content
sub	User UUID
email	User email
role	USER or ADMIN
exp	24h default (JWT_EXPIRATION_MS)



Google OAuth redirect flow



8. Content & Blog Design



Blog rendering

- **JSON blogs** (seeded): `BlogRenderer.jsx` renders structured blocks (headings, paragraphs, stickers, TOC)
- **Admin blogs**: plain text content with paragraph breaks
- **Articles**: read-only curated content, seeded on first boot

9. API Design Principles

- **RESTful resources** — nouns (`/api/blogs`), HTTP verbs for actions
- **Slug-based identity** — human-readable URLs for blogs and articles
- **Stateless sessions** — JWT in Authorization header, no server-side session store
- **Consistent errors** — `{ "message": "..."}` via `ApiExceptionHandler`
- **Public read, authenticated write** — content discovery without login

10. Non-Functional Requirements

Attribute	Target	Current implementation
Availability	Best-effort on free tier	Render cold starts; health checks
Scalability	Horizontal API scaling	Stateless API; connection pooler
Security	HTTPS, JWT, OAuth	Spring Security, BCrypt, CORS
Maintainability	Layered backend, componentized FE	Controller → Service → Repository
Portability	Cloud-agnostic API	Docker + env-driven config
Observability	Health endpoint	GET /api/health, Render logs

UI design system

Element	Choice
Typography	Cormorant Garamond (headings) + DM Sans (body)
Palette	Cream background (#f7f4ee), forest green accents (#2d5a3d)
Layout	People Grove–inspired editorial dashboard
Responsive	Mobile-friendly grid and navigation

11. Design Decisions (ADR Summary)

Decision	Rationale	Trade-off
React SPA on Vercel	Fast deploys, CDN, free tier	No SSR; SEO limited
Spring Boot on Render Docker	No native Java on Render	Slow cold starts on free tier
Supabase pooler not direct DB	Render is IPv4-only	Extra env config
JWT not server sessions	Stateless, simple SPA auth	No instant token revoke
Flyway migrations	Versioned schema, reproducible	Migration discipline required
JSON blocks for seed blogs	Rich layout without CMS	Two blog render paths
Google OAuth optional profile	Local dev without Google creds	Profile activation via endpoint